

APPENDIX A
THEORETICAL PROOFS

A. Proof of Theorem 1

Proof. We consider standard gradient updates:

$$\begin{aligned}\mathbf{w}_{t+1}^\tau &= \mathbf{w}_t^\tau - \eta \nabla \mathcal{L}^\tau(\mathbf{w}_t^\tau), \\ \mathbf{w}_{t+1}^s &= \mathbf{w}_t^s - \eta \nabla \mathcal{L}^s(\mathbf{w}_t^s).\end{aligned}\quad (17)$$

Then, the difference between the two trajectories at step $t+1$ can be calculated as:

$$\begin{aligned}\mathbf{w}_{t+1}^s - \mathbf{w}_{t+1}^\tau &= (\mathbf{w}_t^s - \eta \nabla \mathcal{L}^s(\mathbf{w}_t^s)) - (\mathbf{w}_t^\tau - \eta \nabla \mathcal{L}^\tau(\mathbf{w}_t^\tau)) \\ &= (\mathbf{w}_t^s - \mathbf{w}_t^\tau) - \eta (\nabla \mathcal{L}^s(\mathbf{w}_t^s) - \nabla \mathcal{L}^\tau(\mathbf{w}_t^\tau)).\end{aligned}\quad (18)$$

Taking the norm, we have:

$$\|\mathbf{w}_{t+1}^s - \mathbf{w}_{t+1}^\tau\| \leq \|\mathbf{w}_t^s - \mathbf{w}_t^\tau\| + \eta \|\nabla \mathcal{L}^s(\mathbf{w}_t^s) - \nabla \mathcal{L}^\tau(\mathbf{w}_t^\tau)\|.\quad (19)$$

Finally, we apply Eq. (19) recursively from $t=0$ to $t=T-1$, noting that $\mathbf{W}_0^s = \mathbf{W}_0^\tau$, we obtain:

$$\|\mathbf{w}_T^s - \mathbf{w}_T^\tau\| \leq \eta \sum_{k=0}^{T-1} \|\nabla \mathcal{L}^s(\mathbf{w}_k^s) - \nabla \mathcal{L}^\tau(\mathbf{w}_k^\tau)\|.\quad (20)$$

□

B. Proof of Theorem 2

Proof. As the gradient matching objective is commonly calculated for each class separately, we define the class-wise loss for $\mathcal{T}$ and $\mathcal{S}$ as:

$$\begin{aligned}\mathcal{L}_{\text{GM}} &= \sum_{i=0}^{n-1} \left\| \frac{1}{|n_i|} \nabla \mathcal{L}_i^\tau(\mathbf{W}) - \frac{1}{|n'_i|} \nabla \mathcal{L}_i^s(\mathbf{W}) \right\|^2 \\ &= \sum_{i=0}^{C-1} \left\| \frac{1}{|n_i|} (\Phi_i^\top \Phi_i \mathbf{W} - \Phi_i^\top Y_i) - \frac{1}{|n'_i|} (\Phi_i'^\top \Phi_i' \mathbf{W} - \Phi_i'^\top Y_i') \right\|^2 \\ &= \sum_{i=0}^{C-1} \left\| \left(\frac{1}{|n_i|} \Phi_i^\top \Phi_i - \frac{1}{|n'_i|} \Phi_i'^\top \Phi_i' \right) \mathbf{W} - \left(\frac{1}{|n_i|} \Phi_i^\top Y_i - \frac{1}{|n'_i|} \Phi_i'^\top Y_i' \right) \right\|^2 \\ &\leq \sum_{i=0}^{C-1} \left\| \frac{1}{|n_i|} \Phi_i^\top Y_i - \frac{1}{|n'_i|} \Phi_i'^\top Y_i' \right\|^2 + \sum_{i=0}^{C-1} \left\| \frac{1}{|n_i|} \Phi_i^\top \Phi_i - \frac{1}{|n'_i|} \Phi_i'^\top \Phi_i' \right\|^2 \|\mathbf{W}\|^2 \\ &= \|\mathbf{P}\Phi(X) - \mathbf{P}'\Phi(X')\|^2 + \sum_{i=0}^{C-1} \left\| \frac{1}{|n_i|} \Phi_i^\top \Phi_i - \frac{1}{|n'_i|} \Phi_i'^\top \Phi_i' \right\|^2 \|\mathbf{W}\|^2.\end{aligned}\quad (21)$$

This decomposition reveals that distribution matching, when combined with second-order embedding alignment, provides an upper bound for class-wise gradient matching. □

C. Proof of Theorem 3

Proof. Recall that MCKP is as follows: Given m mutually disjointed groups, where group G_i contains a set of K_i items $\{x_{i,1}, x_{i,2}, \dots, x_{i,K_i}\}$. Each item $x_{i,k} \in G_i$ has a value $v_{i,k} \in \mathbb{R}_{>0}$ and a weight $w_{i,k} \in \mathbb{Z}_{>0}$. The capacity of the knapsack is W . MCKP aims to select exactly one item from each group such that the total weight $\sum_i w_{i,\sigma(i)} \leq W$ is within the budget and the total value $\sum_i v_{i,\sigma(i)}$ is maximized. Here, the mapping $\sigma : \{1, 2, \dots, m\} \rightarrow \mathbb{Z}_{>0}$ specifies the index of the selected item in each group, i.e., $x_{i,\sigma(i)} \in G_i$ denotes the item chosen for group i . Based on this, we construct an instance of class-adaptive cluster allocation as: there are m classes of samples which corresponds to the number of groups in MCKP. The candidate set of cluster numbers for class i is denoted as $\{n'_{i,1}, n'_{i,2}, \dots, n'_{i,K_i}\}$, where $n'_{i,k} = k$

and $K_i = \max\{N' - (C-1), n_i\}$. This defines the discrete search space for each class i , which is structurally equivalent to the item set G_i in MCKP. Each element in this set corresponds to an item in MCKP, where the sample count $n'_{i,k}$ also plays a role of the item's weight $w_{i,k}$. Assume that we enumerate all candidate $n'_{i,k}$, and compute corresponding optimal clustering loss $\mathcal{L}(\{S_j^{i,k}\}^*)$, which corresponds to the negative value $-v_{i,k}$ in MCKP. The total cluster budget is set to W , satisfying that $\sum_i w_{i,\sigma(i)} = W$. From this instance, we can derive a perfect matching for the MCKP problem. This completes the NP-hardness proof. □

D. Proof of Theorem 4

Proof. In Algorithm 1, the initialization stage incurs its main cost in the ClassWiseClustering (Algorithm 2), invoked at line 6 of Algorithm 1, where each class i performs RunK-Means on $X_i \in \mathbb{R}^{n_i \times F}$ for I iterations. This step requires $\mathcal{O}(n_i n'_i I F)$ time, where $n'_i = \lfloor n_i \cdot r \rfloor$. Hence, the overall initialization complexity is $\mathcal{O}(I F \sum_{i=0}^{C-1} n_i \lfloor n_i r \rfloor)$. After initialization, the algorithm iteratively alternates between SoftAllocate (Algorithm 3, invoked at line 11 of Algorithm 1), which runs in constant time, and ClassWiseClustering (line 12), which dominates the iteration cost. In the worst case, clustering must be recomputed for all classes in each iteration. Let $\bar{n}' = \frac{1}{C} \sum_{i=0}^{C-1} n'_i = \frac{N'}{C}$ denote the average number of clusters per class. Then, over T iterations, the total cost of the search phase is $\mathcal{O}(T \sum_{i=0}^{C-1} n_i \bar{n}' I F) = \mathcal{O}(T \cdot \frac{N N' I F}{C})$. After convergence, Algorithm 1 performs RunKMeans(X_i, n'_i) for each class (line 22) to obtain the final partitions $\{\{S_j^i\}^*\}$. This post-processing cost is negligible when T is large. The time complexity of the search over T steps is $\mathcal{O}(T \cdot \frac{N N' I F}{C})$. Overall, the time complexity of HFILS is $\mathcal{O}(I F \sum_{i=0}^{C-1} n_i \cdot \lfloor n_i \cdot r \rfloor + T \cdot \frac{N N' I F}{C}) = \mathcal{O}(T \cdot \frac{N N' I F}{C})$. □

APPENDIX B
RELATED WORK

A. Coreset selection

Coreset selection methods aim to select a representative subset that achieves comparable model performance to training on the full dataset. Existing methods primarily rely on intermediate objectives that enforce distributional consistency between the subset and the original dataset [57], [58], [64]–[66]. Representative approaches include herding [57], which greedily adds samples to minimize the feature-space distance between the centers of the subset and the full dataset, and k -center [58], which iteratively selects samples to maximize spatial coverage. More recently, some methods have leveraged training signals such as gradients, loss values, or forgetting events to guide the selection of informative samples [16], [48], [67]–[69]. Despite their effectiveness, coreset selection methods share inherent limitations: since they can only select a small subset of existing samples, they are constrained by the original dataset and cannot generate new, potentially more representative instances [19]–[21]. Therefore, they fail to fully exploit the rich information in the full dataset, inevitably

leading to substantial information loss [67]. This problem is further amplified when informative signals are broadly distributed rather than concentrated in a few samples [48].

APPENDIX C OVERALL FRAMEWORK

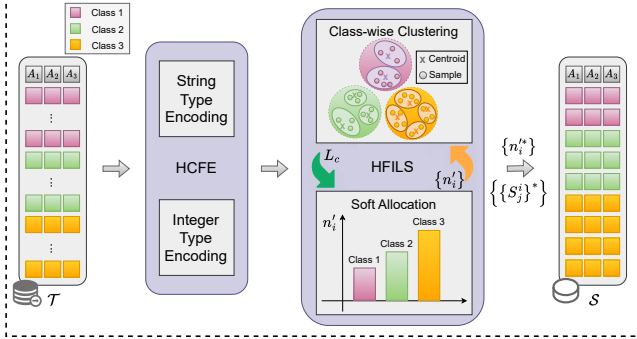


Fig. 5: Overall framework of C^2TC .

APPENDIX D ALGORITHMS

A. Class-wise Clustering Algorithm

Algorithm 2 illustrates the CW-Clus procedure. For each class, the clustering loss is either computed via K-means (lines 3-8) or retrieved from the cached loss matrix $\mathbf{M}$ when the corresponding entry already exists (i.e., the condition in line 4 is not met). The total clustering loss $\mathcal{L}_c$ is then accumulated over all classes (line 9) and returned as the overall objective value (line 10).

Algorithm 2: ClassWiseClustering

Input : Original dataset $\mathcal{T} = (X, Y)$, number of classes C , current cluster allocation $\{n'_i\}$, total cluster count N' , adaptive exponent γ , loss matrix $\mathbf{M}$
Output : Total clustering loss $\mathcal{L}_c$

```

1  $X_i \leftarrow \{x \in X \mid y(x) = i\}$  for each  $i \in \{0, 1, \dots, C-1\}$ ;
2  $\mathcal{L}_c \leftarrow 0$ ;
3 for  $i \in \{0, 1, \dots, C-1\}$  do
4   if  $\mathbf{M}[i, n'_i] = \infty$  then
5     // Compute class-wise clustering loss
6      $\text{SSD}, \{S_j^i\}^* \leftarrow \text{RunKMeans}(X_i, n'_i)$ ;
7      $n_i \leftarrow |X_i|$ ;
8     // Compute class-wise scale factor
9      $w \leftarrow \frac{1}{n_i^\gamma}$ ;
10     $\mathbf{M}[i, n'_i] \leftarrow \text{SSD} \times w$ ;
11   $\mathcal{L}_c \leftarrow \mathcal{L}_c + \mathbf{M}[i, n'_i]$ ;
12 return  $\mathcal{L}_c$ 
```

B. Soft Allocate Algorithm

To facilitate dynamic local search and better approximate the optimum of in Eq. (12), we design a neighborhood scanning strategy based on composite reallocation moves. In each move, the algorithm decreases the cluster count of one source class while simultaneously increasing those of multiple target classes, thereby realizing adaptive redistribution across classes. Instead of a rigid one-to-one adjustment where a

single class gains clusters strictly at the expense of another, our approach employs a *softer reallocation mechanism* that enables smoother, more flexible, and better-balanced redistribution across multiple classes.

The soft allocation procedure is illustrated in Algorithm 3. It first selects a source class c_{src} with more than one cluster (line 1) and constructs the target class set $\mathcal{C}_{\text{tgt}}$, which contains classes that can still receive additional clusters within their upper limits (lines 2-4). A real step size s is then randomly sampled from $[1, s_{\text{max}}]$ and bounded by the transferable capacity of c_{src} (lines 5-6), introducing controlled stochasticity that promotes diverse yet feasible adjustments. After subtracting s clusters from the source class (line 7), the reallocation proceeds according to the remaining capacities of target classes. For each class in $\mathcal{C}_{\text{tgt}}$, the algorithm estimates its residual capacity r_c (lines 8-9) and determines a provisional allocation a_c proportional to its relative ratio $\frac{r_c}{r_{\text{tot}}}$ (lines 11-12), guided by the elbow heuristic that prioritizes classes expected to yield higher clustering gains. To handle rounding effects, the remaining clusters δ are sequentially assigned to classes with larger residual capacities until all clusters are fully allocated (lines 13-19). Finally, the per-class cluster counts n'_c are updated (lines 20-21) to reflect the completed redistribution, resulting in an updated allocation $\{n'_i\}$ (line 22).

Algorithm 3: SoftAllocate

Input : Per-class sample sizes $\{n_i\}$, current cluster allocation $\{n'_i\}$, number of classes C , total cluster count N' , current max step size s_{max}
Output : Updated per-class cluster sizes $\{n'_i\}$

```

1  $c_{\text{src}} \sim \text{Uniform}(\{i \mid n'_i > 1\})$ ;
2 for  $i \in \{0, 1, \dots, C-1\}$  do
3    $n'_{i, \text{max}} \leftarrow \min\{N' - (C-1), n_i\}$ ;
4  $\mathcal{C}_{\text{tgt}} = \{i \mid n'_i < n'_{i, \text{max}} \wedge i \neq c_{\text{src}}\}$ ;
5 // Randomize a real step size
6  $s \sim \text{Uniform}(\{1, 2, \dots, s_{\text{max}}\})$ ;
7  $s \leftarrow \min(s, n'_{c_{\text{src}}} - 1)$ ;
8  $n'_{c_{\text{src}}} \leftarrow n'_{c_{\text{src}}} - s$ ;
9 // Allocate by remaining capacity ratio
10 for each  $c \in \mathcal{C}_{\text{tgt}}$  do
11    $r_c \leftarrow n'_{c, \text{max}} - n'_c$ ;
12  $r_{\text{tot}} \leftarrow \sum_{c \in \mathcal{C}_{\text{tgt}}} r_c$ ;
13 for each  $c \in \mathcal{C}_{\text{tgt}}$  do
14    $a_c \leftarrow \left\lfloor s \cdot \frac{r_c}{r_{\text{tot}}} \right\rfloor$ ;
15 // Distribute unassigned clusters
16  $\delta \leftarrow s - \sum_{c \in \mathcal{C}_{\text{tgt}}} a_c$ ;
17 sort  $\mathcal{C}_{\text{tgt}}$  by descending  $r_c$ ;
18 for each  $c \in \mathcal{C}_{\text{tgt}}$  do
19   if  $\delta = 0$  then
20     break;
21   if  $n'_c + a_c + 1 \leq n'_{c, \text{max}}$  then
22      $a_c \leftarrow a_c + 1$ ;  $\delta \leftarrow \delta - 1$ ;
23 for each  $c \in \mathcal{C}_{\text{tgt}}$  do
24    $n'_c \leftarrow n'_c + a_c$ ;
25 return  $\{n'_i\}$ 
```

A. Datasets

To evaluate the effectiveness and efficiency of our approach, we conduct experiments on 10 publicly available tabular datasets from the widely adopted OpenML benchmarks [56]. These datasets cover diverse real-world domains, including but not limited to energy forecasting, income prediction, and flight delay analysis, and encompass heterogeneous data types, such as numerical and categorical features. Their sizes range from the order of 10^4 to 10^6 instances, ensuring a comprehensive evaluation across small- to large-scale settings. In addition, we include datasets with different label distributions, reflecting varying degrees of class imbalance commonly observed in real-world data. To measure this imbalance, we adopt the widely used metric known as the ratio of minimum to maximum class frequency [59], where smaller values indicate a higher degree of imbalance. For all datasets, we randomly split them for 80%/10%/10% as train/validation/test sets. Table I provides the detailed statistics of all datasets.

B. Baselines

We compare our method against a set of representative baselines, including coreset selection approaches (Random, Herding [57], and K-Center [58]) and dataset condensation methods (DM [21], GM [19], and MTT [20]).

- **Random.** Random selects a subset of training samples uniformly at random from the original data as the coreset.
- **Herding** [57]. Herding is a greedy algorithm that iteratively selects samples closest to the cluster center.
- **K-Center** [58]. K-Center picks multiple center points such that the largest distance between a data point and its nearest center point is minimized.
- **DM** [21]. DM learns condensed dataset by matching the feature embeddings between real and synthetic samples.
- **GM** [19]. GM proposes to infer the synthetic dataset by matching the gradients of a model trained on synthetic dataset to that on the original dataset.
- **MTT** [20]. MTT builds a condensation dataset to match the parameter trajectory of full dataset training.

C. Evaluation Metrics

To evaluate the classification performance of models trained on condensed datasets, we adopt two widely used metrics: *Accuracy* and *Macro-F1*. Accuracy is defined as the proportion of correctly predicted instances:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad (22)$$

where TP , TN , FP , and FN denote true positives, true negatives, false positives, and false negatives, respectively. To further provide a balanced view of classification performance across all classes, we utilize Macro-F1, which is the harmonic mean of precision and recall, computed independently for each class and then averaged:

$$\text{Macro-F1} = \frac{1}{C} \sum_{c=0}^{C-1} \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}, \quad (23)$$

where $\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}$ and $\text{Recall}_c = \frac{TP_c}{TP_c + FN_c}$.

D. Implementation Details

The pipeline of dataset condensation has two stages: (1) **Condensation step**, where we apply the DC methods on the training set to obtain small-scale synthetic data. (2) **Evaluation step**, where we train deep models on the condensed data from scratch and then evaluate their performance on the test set of the original data. For existing DC methods (i.e., DM, GM, and MTT) that requires model training, we use FT-Transformer [11], an effective and widely adopted tabular learning method, as the relay model. Our effectiveness experiments consider three reduction rates (1%, 0.1%, and 0.01%), with 0.1% used as the default setting in subsequent experiments unless stated otherwise. For C²TC, we set the maximum iteration to $T = 1000$, the tolerance to $\epsilon = 0.01$, and the early-stopping patience to $p = 10$. The adaptive weighting exponent γ is tuned over $\{0.25, 0.5, 0.75, 1.0\}$. Besides, we tune the step decay l in $\{0.1, 0.3, 0.5, 0.7, 0.9\}$. For all baselines, we report results using the official implementations with default hyperparameters specified in the original papers, and apply label encoding for categorical features. Methods that exceed the runtime limit (24 hours during condensation, or 5 days for expert trajectories generation during pre-processing in MTT) are denoted as OOT (out of time), while those that fail due to memory allocation errors are denoted as OOM (out of memory). The experiments are conducted on a Linux server with an Intel(R) Xeon(R) Gold 6330 CPU, a NVIDIA A800 GPU, and 128GB of RAM. To eliminate randomness, each experiment is repeated five times, and the average results with standard deviation are reported.

E. Efficiency Evaluation

Condensation time over all datasets. In Figure 6, we first compare the condensation time of our approach with state-of-the-art DC baselines across all tabular datasets. As can be seen, C²TC exhibits significantly lower condensation time compared to existing DC methods, consistently achieving at least 2 orders of magnitude speedup across all datasets. For example, on the largest dataset MI, C²TC only takes 39.2 seconds during condensation, while MTT requires over 78 hours to complete the synthesis process and GM fails to finish within the time limit. While on the high-dimensional dataset (EP), existing DC methods either suffer from OOT or OOM issues and are therefore not included in the figure, whereas C²TC remains scalable, completing condensation in under 72 seconds. This remarkable efficiency gain can be attributed to our training-free design, which avoids the Siamese architecture and gradient-based iterative updates adopted by prior methods, thereby eliminating major computational bottlenecks.

Condensation time by varying N and F . In this experiment, we study the scalability of condensation methods with respect to sample size N and feature dimension F on the MI and EP datasets, respectively. Specifically, on MI we vary the sample

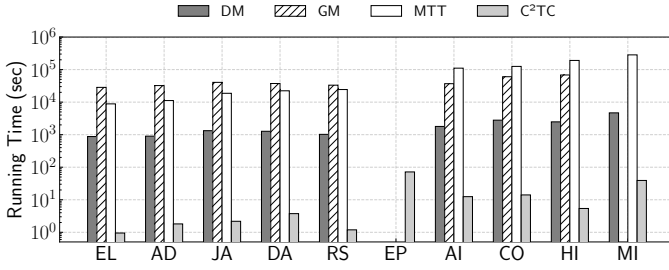


Fig. 6: Condensation time comparison.

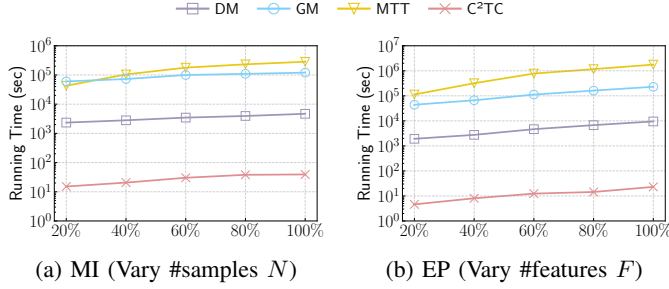


Fig. 7: Condensation time by varying N and F .

size from 20% to 80% to form new datasets under the default reduction rate (0.1%), whereas on EP we fix the sample size and vary the feature dimension from 20% to 80% under a reduction rate of 0.01%. Figure 7(a) and Figure 7(b) show the condensation time of C^2TC and the three DC methods with varying N and F , respectively. As observed, C^2TC outperforms the other algorithms by a significant margin and scales well. For example, when the percentage of N is 20% in Figure 7(a), the condensation times of DM, GM, and MTT are 2340.2 seconds, 6.0×10^4 seconds, and 4.3×10^4 seconds, respectively, whereas C^2TC requires only 15.2 seconds. When the percentage of N increases to 80%, C^2TC completes in 38.1 seconds, while DM, GM, and MTT scale up to 3943.7, 1.1×10^5 , and 2.3×10^5 seconds, respectively. In Figure 7(b), we find that C^2TC consistently achieves at least 2 orders of

magnitude speedup over existing DC methods across different ratios of F on the EP dataset. As F increases from 20% to 80%, the condensation time of C^2TC rises only slightly from 4.6 seconds to 14.4 seconds. In contrast, the runtime of the fastest DC baseline, DM, grows rapidly from 1920.7 seconds to 6749.5 seconds. These results highlight the superior scalability of C^2TC , in line with our complexity analysis.

F. Ablation Study

Analysis of label allocation strategies. To validate the effectiveness of our adaptive allocation strategy, which is developed on top of our proposed training-free objective to appropriately determine the per-class cluster number n'_i , we replace it with two static label allocation strategies, Ratio and FIPC. Ratio allocates clusters proportionally to the original class distribution, whereas FIPC assigns an equal number of synthetic samples to each class. As shown in Table VII, our class-adaptive strategy consistently achieves superior performance on both metrics across all datasets. For instance, it improves accuracy by 5.3% and Macro-F1 by 5.8% on EP, and by 4.0% and 5.9% on CO, demonstrating strong robustness across different class distribution scenarios. This is because our C^2TC dynamically balances the influence of majority and minority classes, leading to a better trade-off between overall accuracy and per-class performance. Compared with FIPC, Ratio achieves higher accuracy but lower Macro-F1 on imbalanced datasets (AD, CO, MI), as it allocates more clusters to majority classes and overlooks minority ones. FIPC, on the other hand, enforces uniform allocation across classes, improving minority-class performance at the cost of overall accuracy dominated by majority classes.

TABLE VII: Comparison of label allocation strategies by Accuracy (Acc) and Macro-F1 (MF1).

Strategy	EL		AD		EP		CO		MI	
	Acc	MF1	Acc	MF1	Acc	MF1	Acc	MF1	Acc	MF1
Ratio	63.0	62.9	80.4	69.7	69.9	69.0	65.6	33.1	53.5	20.9
FIPC	63.0	62.9	74.2	70.7	69.9	69.0	47.3	40.5	34.2	20.8
C^2TC	64.5	63.5	81.5	70.8	75.2	74.8	69.6	46.4	53.6	21.5